

Digital Systems Design

(BECE102L)

Dr. Sukanta Kumar Tulo
SENSE, VIT Vellore

Course Objectives

1. Provide an understanding of Boolean algebra and logic functions.
2. Develop the knowledge of combinational and sequential logic circuit design.
3. Design and model the data path circuits for digital systems.
4. Establish a strong understanding of programmable logic.
5. Enable the student to design and model the logic circuits using Verilog HDL.

Course Outcome

- 1. Optimize the logic functions using Boolean principles and K-map.**
- 2. Model the Combinational and Sequential logic circuits using Verilog HDL.**
- 3. Design the various combinational logic circuits and data path circuits.**
- 4. Analyze and apply the design aspects of sequential logic circuits.**
- 5. Analyze and apply the design aspects of Finite state machines.**
- 6. Examine the basic architectures of programmable logic devices.**

Syllabus

Module 1: Digital Logic

Module 2: Verilog HDL

Module 3: Design of Combinational Logic Circuits

Module 4: Design of data path circuits

Module 5: Design of Sequential Logic Circuits

Module 6: Design of FSM

Module 7: Programmable Logic Devices

Recommended Books

Textbook(s)

M. Morris Mano and Michael D. Ciletti, Digital Design: With an Introduction to the Verilog HDL and System Verilog, 2018, 6th Edition, Pearson Pvt. Ltd.

Recommended Books

Reference Books

1. Ming-Bo Lin, Digital Systems Design and Practice: Using Verilog HDL and FPGAs, 2015, 2nd Edition, Create Space Independent Publishing Platform.
2. Samir Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, 2009, 2nd edition, Prentice Hall of India Pvt. Ltd.
3. Stephen Brown and Zvonko Vranesic, Fundamentals of Digital Logic with Verilog Design, 2013, 3rd Edition, McGraw-Hill Higher Education.

Module:1

Digital Logic

Dr. Sukanta Kumar Tulo
SENSE, VIT Vellore

Module:1 Digital Logic Contents

- Boolean Algebra: Basic definitions,
- Basic Theorems and Properties of Boolean Algebra,
- Boolean Functions,
- Canonical and Standard Forms,
- Simplification of Boolean functions,
- Gate-Level Minimization: The Map Method: K-map up to 4 variable,
- Product of Sums and Sum of Products Simplification,
- NAND and NOR Implementation,
- Logic Families: Digital Logic Gates, TTL and CMOS logic families.

Basic Definitions

- **Boolean algebra:** a set of elements, a set of operators, and a number of axioms and postulates
- **Axiom:** a statement which is regarded as being established or accepted
- **Postulates:** basic assumptions from which it is possible to deduce the rules, theorems, and properties of the system

Axioms, Postulates and Theorems of Boolean Algebra

- $x + 0 = x$

- $x + 1 = 1$

- $x + x = x$

- $x + x' = 1$

- $x \cdot 0 = 0$

- $x \cdot 1 = x$

- $x \cdot x = x$

- $x \cdot x' = 0$

Axioms, Postulates and Theorems of Boolean Algebra

- Commutative law

$$x + y = y + x$$

$$x y = y x$$

- Associative law

$$x + (y + z) = (x + y) + z$$

$$x (y z) = (x y) z$$

- Distributive law

$$x (y + z) = x y + x z$$

$$x + y z = (x + y)(x + z)$$

- DeMorgan Theorem

$$(x + y)' = x' y'$$

$$(x y)' = x' + y'$$

- Absorption Theorem

$$x + x y = x$$

$$x (x + y) = x$$

- Involution Theorem

$$(x')' = x$$

Logic Gates

Logic Gates

Boolean Functions

- Boolean function: an algebraic expression consisting of binary variables, the constants 0 and 1, and the logic operation symbols.
- Function results in binary 0 or 1
- A Boolean function can be represented in a truth table.
- The number of rows in the truth table is 2^n , where n is the number of variables in the function.

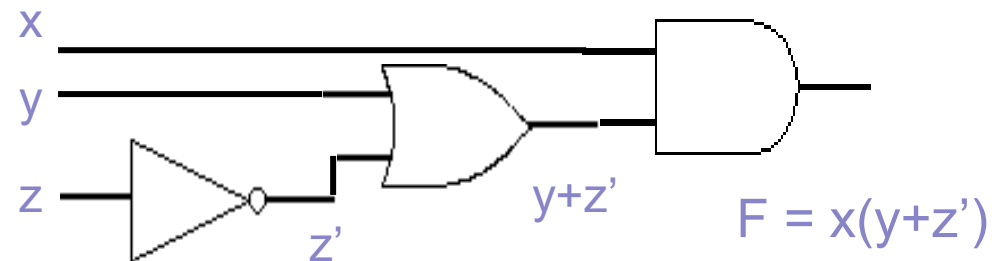
Truth Table

$$F = x (y + z')$$

Truth Table

Input					Output
x	y	z	z'	(y + z')	F
0	0	0	1	1	0
0	0	1	0	0	0
0	1	0	1	1	0
0	1	1	0	1	0
1	0	0	1	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	0	1	1

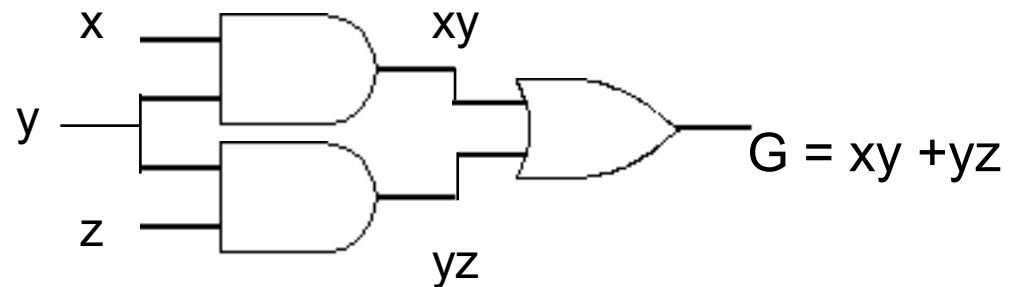
Logic diagram



Truth Table

$$G = xy + yz$$

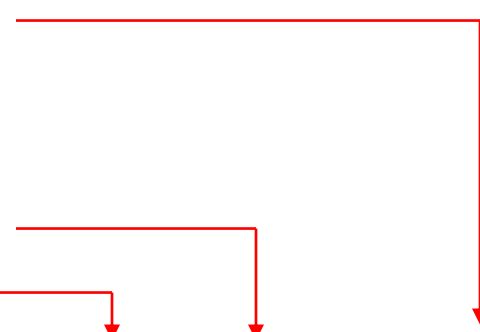
x	y	z	xy	yz	G
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	0	0	0
0	1	1	0	1	1
1	0	0	0	0	0
1	0	1	0	0	0
1	1	0	1	0	1
1	1	1	1	1	1



Truth Table to Expression and to Circuit

x	y	z	G
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

- Each row with output of **1** becomes a **product term**
- Find the **Sum** of product terms together.


$$xyz + xyz' + x'yz$$

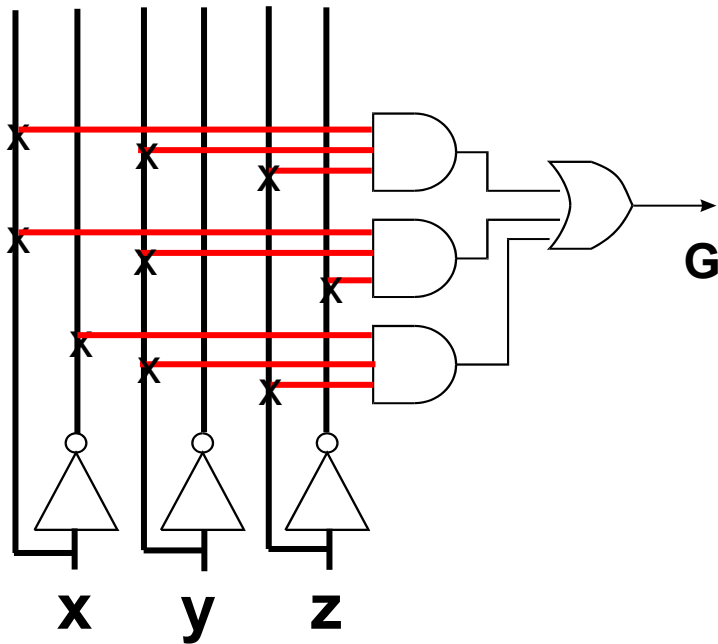
Expression

$$G = xyz + xyz' + x'yz$$

*Any Boolean Expression can be represented in **Sum Of Products (SOP)** form*

Truth Table to Expression and to Circuit

$$xyz + xyz' + x'yz \quad (\text{SOP})$$



Circuit

Simplification of Boolean Functions

$$G = xyz + xyz' + x'yz$$

- Is this the smallest possible implementation of this expression?

No!

- By manipulating a Boolean expression using Boolean algebra, a **simpler expression** may be obtained for the same function and thus **reduce the number of gates** in the circuit.
- Designers are motivated to reduce the number of gates because that can **reduce the cost of a circuit**.
- Use Boolean Algebra rules to reduce complexity while preserving functionality.

Simplification of Boolean Functions

$$G = xyz + xyz' + x'yz$$

$$G = xyz + xyz + xyz' + x'yz \quad (\text{Using } x + x = x)$$

$$G = xyz + xyz + xyz' + x'yz$$

$$G = xy(z + z') + yz(x + x') \quad (\text{Using Distributive law: } x(y + z) = xy + xz)$$

$$G = xy.1 + yz.1 \quad (\text{Using } x + x' = 1)$$

$$G = xy + yz \quad (\text{Using } x.1 = x)$$

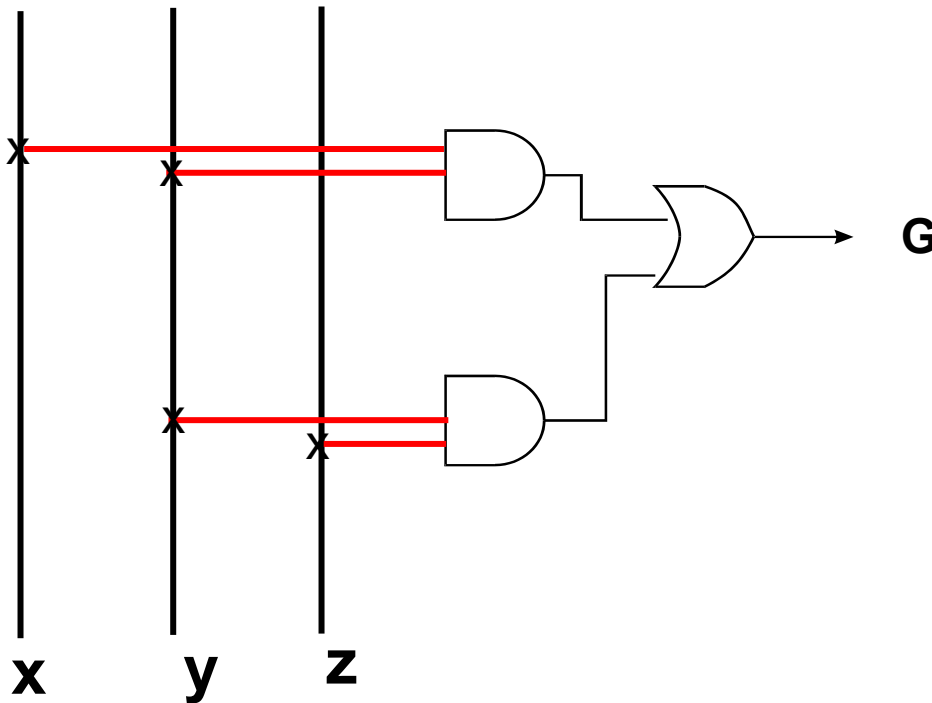
$$\mathbf{G = xyz + xyz' + x'yz = xy + yz}$$

Reduced Hardware Implementation

- Reduced equation requires less hardware!
- Same function implemented!

x	y	z	G
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

$$G = xyz + xyz' + x'yz = \mathbf{xy + yz}$$



Simplification of Boolean Functions

$$F = x' y' z + x' y z + x y'$$

$$= x' y' z + x' y z + x y'$$

$$= x' z (y' + y) + x y' \quad (\text{Using Distributive law: } x (y + z) = x y + x z)$$

$$= x' z + x y' \quad (\text{Using } x + x' = 1)$$

$$H = (x + y) (x + y')$$

$$= x x + x y' + y x + y y'$$

$$= x + x y' + y x + 0$$

$$= x (1 + y' + y)$$

$$= x$$

Simplification of Boolean Functions

$$G = x y + x' z + y z$$

$$= x y + x' z + y z (1)$$

$$= x y + x' z + y z (x + x')$$

$$= x y + x' z + x y z + x' y z$$

$$= x y (1 + z) + x' z (1 + y)$$

$$= x y + x' z$$

Complement of Functions

- Find the complement of the function F

$$F = x (y' z' + y z)$$

$$F' = [x (y' z' + y z)]'$$

$$= x' + (y' z' + y z)'$$
 Applying DeMorgan's theorem

$$= x' + (y' z')' (y z)'$$

$$= x' + (y + z) (y' + z')$$

$$= x' + y y' + y z' + z y' + z z'$$

$$= x' + y z' + y' z$$

Canonical and Standard Forms

- Boolean functions expressed as a sum of minterms (SOP) or product of maxterms (POS) are said to be in **canonical form**.
- To convert a Boolean function into canonical form, identity element theorem has to be used.
- **Standard forms** are like canonical forms, except that not all variables need appear in the individual product (SOP) or sum (POS) terms.

Canonical and Standard Forms

- $F1(x,y,z) = x+y'z$ (SOP)

$$F2(x,y,z) = (x+y)(x'+z) \text{ (POS)}$$

The above two functions are in standard form but not in canonical form.

- $F1(x,y,z) = xyz+xy'z$ (SOP)

$$F2(x,y,z) = (x+y+z)(x'+y+z) \text{ (POS)}$$

The above two functions are in canonical form

- $F1(x,y,z) = x(yz+xy')$

Not in any form

Canonical form

Q) Express the function $F(x,y,z) = x+y'z$ in canonical form.

- Lets consider the first term

x

$$x = x \cdot 1 \cdot 1$$

$$= x (y + y') (z + z')$$

$$= (xy + xy') (z + z')$$

$$= xyz + xyz' + xy'z + xy'z'$$

- Lets consider the second term

$y'z$

$$y'z = y'z (x + x')$$

$$= xy'z + x'y'z$$

$$\text{Finally, } F(x,y,z) = x+y'z = xyz + xyz' + xy'z + xy'z' + xy'z + x'y'z$$

Canonical form

Q) Express the function $F(x,y,z) = xy + x'z$ in POS form and then in canonical form.

$$\begin{aligned} F &= xy + x'z = (xy + x')(xy + z) \\ &= (x + x')(y + x')(x + z)(y + z) \\ &= (x' + y)(x + z)(y + z) \quad \text{POS form} \end{aligned}$$

Canonical form

$$\begin{aligned} x' + y &= x' + y + zz' = (x' + y + z)(x' + y + z') \\ x + z &= x + z + yy' = (x + y + z)(x + y' + z) \\ y + z &= y + z + xx' = (x + y + z)(x' + y + z) \\ F &= (x + y + z)(x + y' + z)(x' + y + z)(x' + y + z') \end{aligned}$$

Minterms

- Each AND combination of variables is called a **minterm**, or a **standard product** (Contains all variables)
- Each variable is primed for 0 and unprimed for 1
- Minterms are represented as m_j

where the j denotes the decimal equivalent of binary number of the minterm

Example

$$A = 0$$

$$B = 0 \quad m_0 = A' \cdot B' \cdot C'$$

$$C = 0$$

Maxterms

- Each OR combination of variables is called a **maxterm**, or a **standard sum**
- Each variable is unprimed for 0 and primed for 1
- Maxterms are represented as M_j

where the j denotes the decimal equivalent of binary number of the maxterm

Example

$$A = 0$$

$$B = 0 \quad M_0 = A + B + C$$

$$C = 0$$

Minterms and Maxterms

Minterms and maxterms for three binary variables

x	y	z	Minterms		Maxterms	
			Term	Designation	Term	Designation
0	0	0	$x'y'z'$	m_0	$x + y + z$	M_0
0	0	1	$x'y'z$	m_1	$x + y + z'$	M_1
0	1	0	$x'yz'$	m_2	$x + y' + z$	M_2
0	1	1	$x'yz$	m_3	$x + y' + z'$	M_3
1	0	0	$xy'z'$	m_4	$x' + y + z$	M_4
1	0	1	$xy'z$	m_5	$x' + y + z'$	M_5
1	1	0	xyz'	m_6	$x' + y' + z$	M_6
1	1	1	xyz	m_7	$x' + y' + z'$	M_7

$$m_j' = M_j$$

Canonical form

- **Canonical Sum-Of-Products:** A function can be represented in canonical SOP as the sum of minterms for which the output is 1 in the truth table.
- For example, It can be presented as

$$F(A, B, C) = m_1 + m_4 + m_5 + m_6 + m_7$$

$$F(A, B, C) = \Sigma(1, 4, 5, 6, 7)$$

- **Canonical Product-Of-Sums:** A function can be represented in canonical POS as the product of maxterms for which the output is 0 in the truth table.
- It can be presented as

$$F = M_0 M_2 M_3 = \Pi(0, 2, 3)$$

Minterms and Maxterms of an Expression

x	y	z	f1	f2
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

- $f_1 = m_1 + m_4 + m_7$

- $f_1 = x'y'z + xy'z' + xyz$

$$(0\ 0\ 0)(1\ 0\ 0)\ (1\ 1\ 1)$$

- $f_1 = \Sigma(1, 4, 7)$

- $f_1 = M_0 \cdot M_2 \cdot M_3 \cdot M_5 \cdot M_6$

- $f_1 = \Pi(0, 2, 3, 5, 6)$

- $f_1 = (x + y + z)(x + y' + z)(x + y' + z')(x' + y + z')(x' + y' + z)$

$$(0\ 0\ 0)(0\ 1\ 0)(0\ 1\ 1)(1\ 0\ 1)(1\ 1\ 0)$$

Minterms and Maxterms of an Expression

Minterms

- $f_2 = m_3 + m_5 + m_6 + m_7$
- $f_2 = \Sigma(3, 5, 6, 7)$
- $f_2 = x'yz + xy'z + xyz' + xyz$

Maxterms

- $f_2 = M_0 M_1 M_2 M_4$
- $f_2 = \Pi(0, 1, 2, 4)$
- $f_2 = (x + y + z)(x + y + z')(x + y' + z)(x' + y + z)$

Conversion Between SOP & POS

$$m_j' = M_j$$

$$F(A, B, C) = \Sigma(1, 4, 5, 6, 7)$$

$$F'(A, B, C) = \Sigma(0, 2, 3) = m_0 + m_2 + m_3$$

$$F = (m_0 + m_2 + m_3)' = m_0' \cdot m_2' \cdot m_3' = M_0 M_2 M_3 = \Pi(0, 2, 3)$$

Convert from SOP to POS form

Q) Convert the SOP to POS

$$A' B' C' + A' B C' + A' B C + A B' C + A B C$$

$$= \Sigma(0, 2, 3, 5, 7)$$

$$= \Pi(1, 4, 6)$$

$$= (A + B + C') (A' + B + C) (A' + B' + C)$$

Simplification of Boolean functions

Gate-Level Minimization: The Map Method (K-map)

- Karnaugh-map (K-map) is an abstract form of Venn diagram, organised as a matrix of squares, where each square represents a minterm
- Adjacent squares always differ by just one literal (so that the identity theorem may apply: $a + a' = 1$)

2-Variable K-maps

- There are 4 minterms for 2 variables (A, B).
- Therefore, there are 4 cells in a 2-variable K-map.

		B	
		0	1
A	0	00 m_0	01 m_1
	1	10 m_2	11 m_3

		B	
		0	1
A	0	$\overline{A}\overline{B}$	$\overline{A}B$
	1	$A\overline{B}$	AB

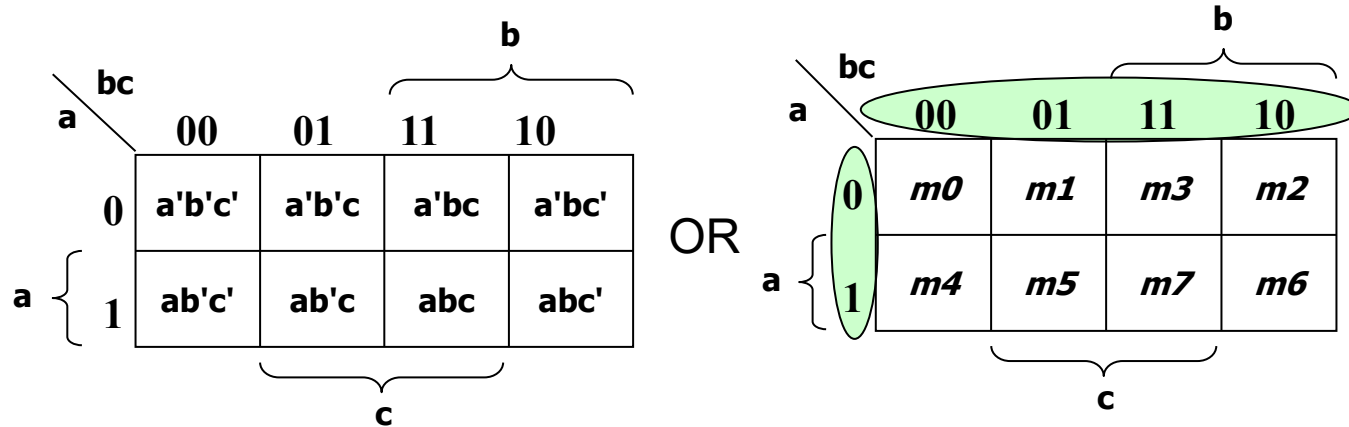
Minterm

		B	
		0	1
A	0	$A + B$	$A + \overline{B}$
	1	$\overline{A} + B$	$\overline{A} + \overline{B}$

Maxterm

3-Variable K-maps

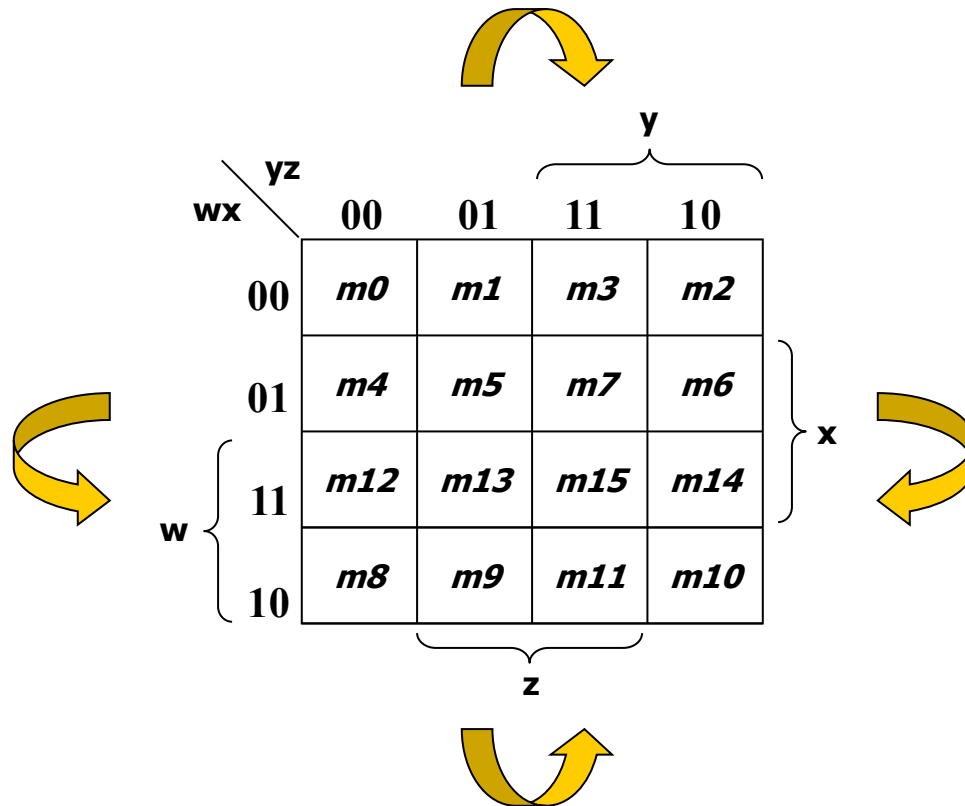
- There are 8 minterms for 3 variables (a, b, c). Therefore, there are 8 cells in a 3-variable K-map.



Above arrangement ensures that minterms of adjacent cells **differ by only ONE literal (variable)**.

4-Variable K-maps

- There are 16 cells in a 4-variable (w, x, y, z) K-map.



There is wrap-around in the K-map

4-Variable K-maps

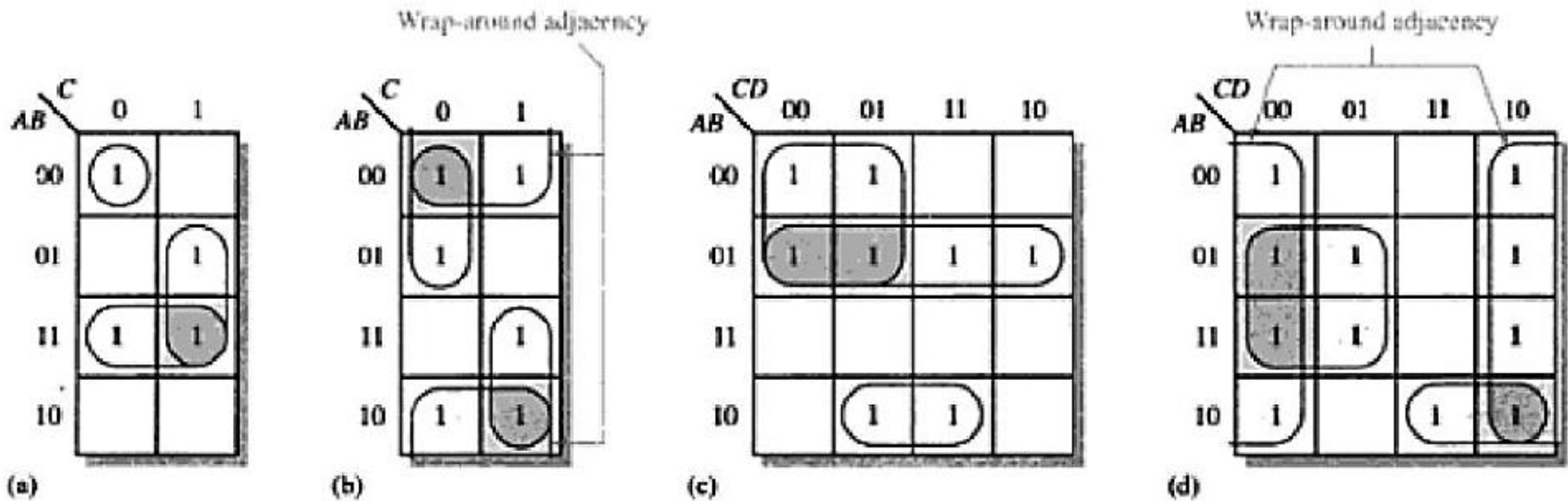
CD AB		CD			
		00	01	11	10
AB	00	0	1	3	2
	01	4	5	7	6
	11	12	13	15	14
	10	8	9	11	10

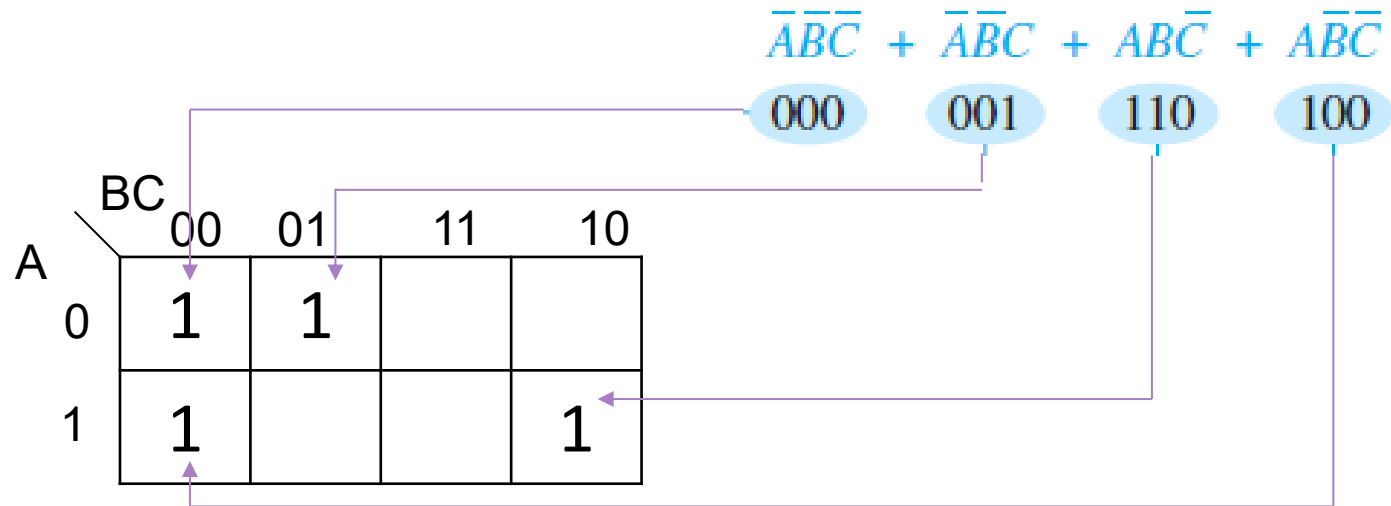
Grouping the Minterms Together

- Make rectangles around groups of one, two, four or eight 1s
- All of the 1s in the map should be included in at least one rectangle
- Do not include any of the 0s
- Each group corresponds to one product term
- Make as few rectangles as possible, to minimize the number of products in the final expression.
- Make each rectangle as large as possible, to minimize the number of literals in each term.
- Rectangles can be overlapped, if that makes them larger.

Grouping

- Example of grouping 1's:

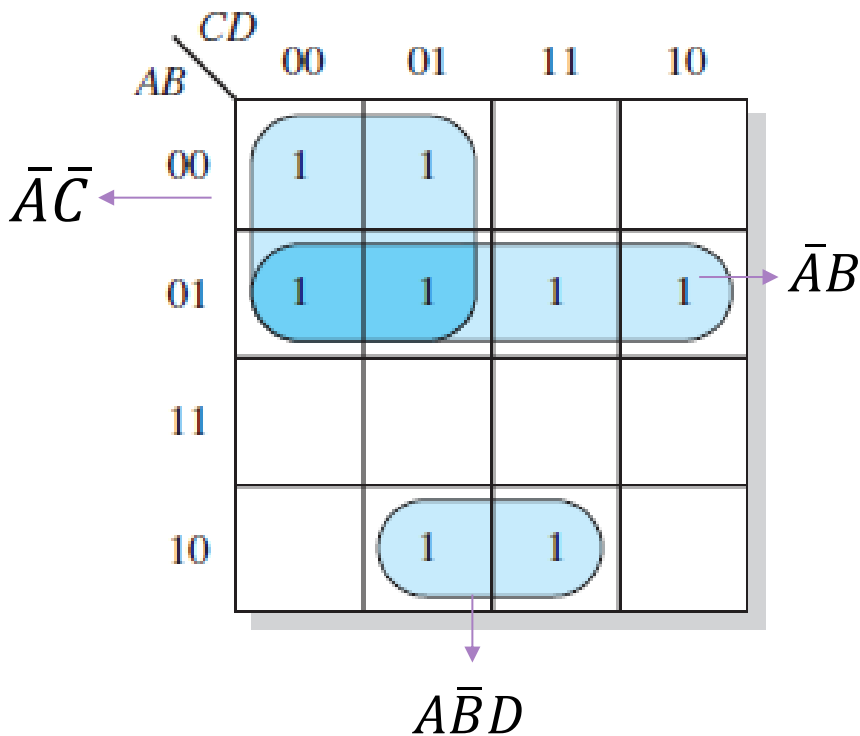




K-map Simplification of SoP Expressions

Simplify to minimum literals using K-Map

$$\bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}\bar{C}D + \bar{A}B\bar{C}\bar{D} + \bar{A}B\bar{C}D + \bar{A}BCD + \bar{A}BC\bar{D} + A\bar{B}\bar{C}D + A\bar{B}CD$$

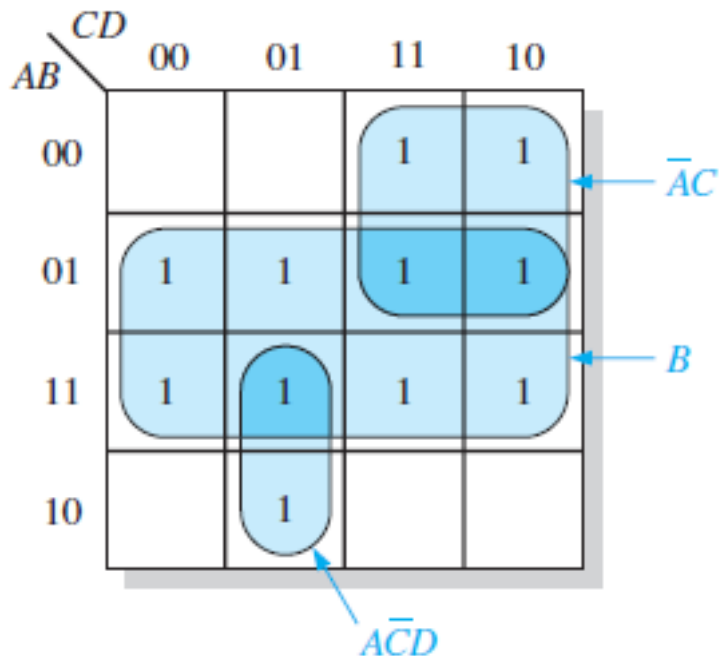


Solution

$$\bar{A}\bar{C} + \bar{A}B + A\bar{B}D$$

Simplify to minimum literals using K-Map

- $\bar{A}\bar{B}CD + \bar{A}\bar{B}C\bar{D} + \bar{A}B\bar{C}\bar{D} + \bar{A}B\bar{C}D + \bar{A}BCD + \bar{A}BC\bar{D} + AB\bar{C}\bar{D} + AB\bar{C}D + ABCD + ABC\bar{D} + A\bar{B}C\bar{D}$

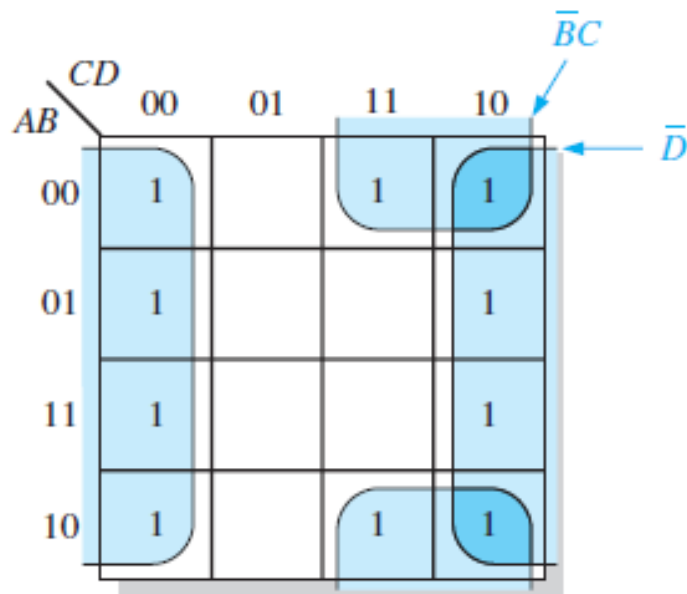


Solution

$$\bar{A}\bar{C} + B + A\bar{C}\bar{D}$$

Simplify to minimum literals using K-Map

$$\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}\overline{D} + A\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}C\overline{D} + A\overline{B}C\overline{D} + \overline{A}\overline{B}C\overline{D} + \overline{A}B\overline{C}\overline{D} + A\overline{B}C\overline{D} + A\overline{B}C\overline{D}$$



$$\overline{B}\overline{C}\overline{D} = (A + \overline{A})\overline{B}\overline{C}\overline{D}$$

$$A\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}\overline{D}$$

Solution

$$\overline{D} + \overline{B}C$$

K-map Simplification of SOP Expressions

- Let's consider simplifying $f(x,y,z) = xy + y'z + xz$
- Convert the expression into a sum of minterms form,
- It can be done using Boolean algebra (identity theorem)
- This can also be done using truth table for the function, and then read off the minterms

Find minterms

- Using algebra

$$\begin{aligned}xy + y'z + xz &= (xy \bullet 1) + (y'z \bullet 1) + (xz \bullet 1) \\&= (xy \bullet (z' + z)) + (y'z \bullet (x' + x)) + (xz \bullet (y' + y)) \\&= (xyz' + xyz) + (x'y'z + xy'z) + (xy'z + xyz) \\&= xyz' + xyz + x'y'z + xy'z \\&= m_1 + m_5 + m_6 + m_7\end{aligned}$$

- Using TT

x	y	z	f(x,y,z)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$\begin{aligned}f(x,y,z) &= x'y'z + xy'z + xyz' + xyz \\&= m_1 + m_5 + m_6 + m_7\end{aligned}$$

K-map

		yz			
		00	01	11	10
x	0		1		
	1		1	1	1

$y'z$

xy

$$f = xy + y'z$$

Simplify $f(x, y, z) = \sum_m(0,1,4,5,6)$ to minimum literals

		yz			
		00	01	11	10
x	0	1	1		
	1	1	1		1

y' xz'

$$f = y' + xz'$$

Don't Care Condition

- In some logic circuits, certain input conditions never occur or we don't bother about the output for those input conditions.
- We call these conditions as Don't care conditions and denote it as X

		CD			
		00	01	11	10
AB	00				
	01				
	11	X	X	X	X
	10		1	X	X

AD

A	B	C	D	X
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	x
1	0	1	1	x
1	1	0	0	x
1	1	0	1	x
1	1	1	0	x
1	1	1	1	x

- Minimize the following function in SOP minimal form using K-Maps:
 $F(A, B, C, D) = m(1, 2, 6, 7, 8, 13, 14, 15) + d(0, 3, 5, 12)$

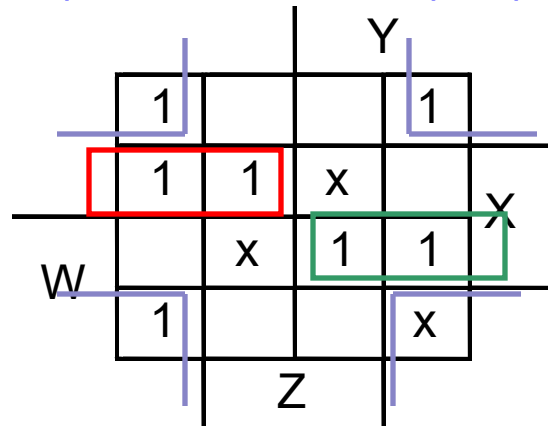
		CD			
		00	01	11	10
AB	00	X	1	X	1
	01		X	1	1
	11	X	1	1	1
	10	1			

$$F = AC'D' + A'D + A'C + AB$$

Don't Care Condition

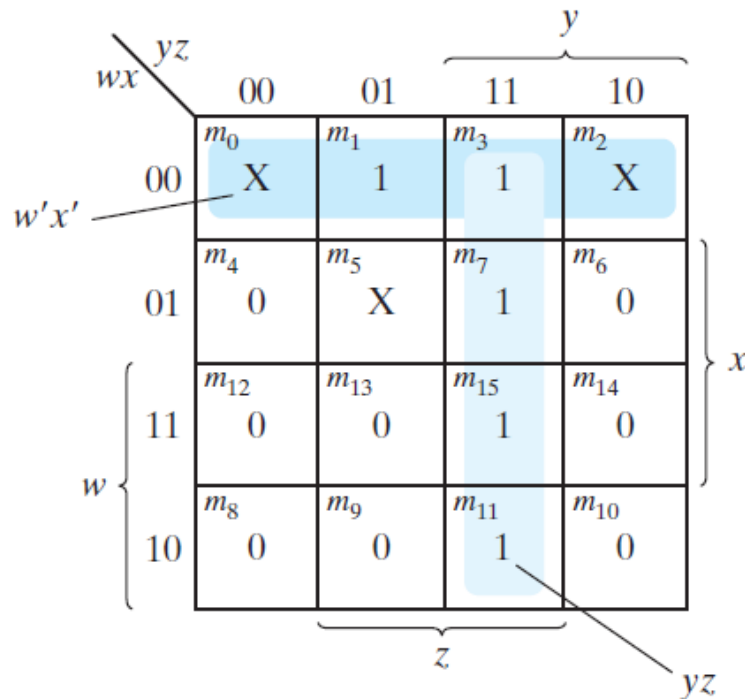
- Find a MSP for:

$$f(w,x,y,z) = \sum m(0,2,4,5,8,14,15), d(w,x,y,z) = \sum m(7,10,13)$$

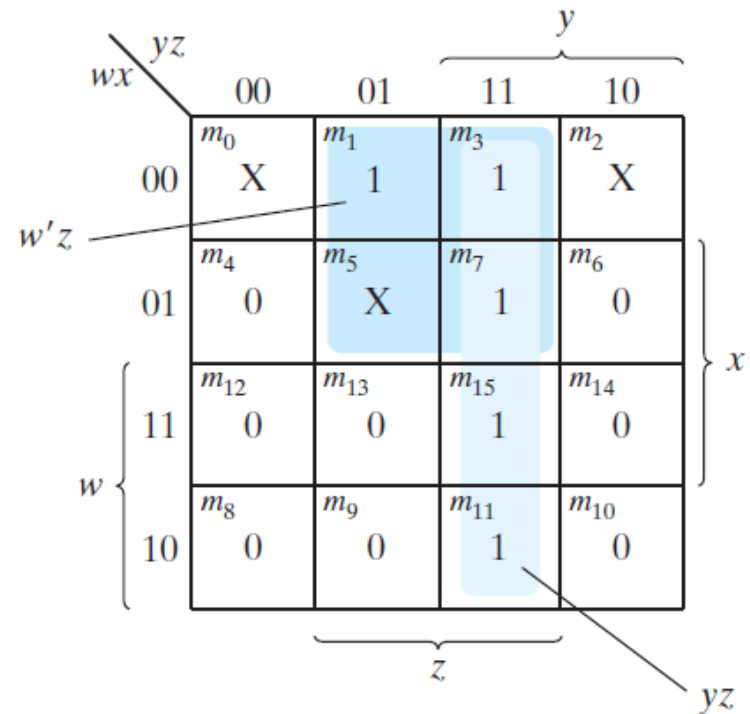


$$f(w,x,y,z) = x'z' + w'xy' + wxy$$

Simplify the Boolean function $F(w, x, y, z) = \Sigma(1, 3, 7, 11, 15)$ which has the don't-care conditions $d(w, x, y, z) = \Sigma(0, 2, 5)$



(a) $F = yz + w'x'$



(b) $F = yz + w'z$

$$F(w, x, y, z) = yz + w'x' = \Sigma(0, 1, 2, 3, 7, 11, 15)$$

$$F(w, x, y, z) = yz + w'z = \Sigma(1, 3, 5, 7, 11, 15)$$

$$\text{Out} = \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}B\overline{C}D + \overline{A}BCD \\ + ABCD + ABC\overline{D} + A\overline{B}\overline{C}\overline{D} + A\overline{B}C\overline{D}$$

A \ B	CD			
	00	01	11	10
00	1	1		
01		1	1	
11			1	1
10	1			1

A \ B	CD			
	00	01	11	10
00	1	1		
01		1	1	
11			1	1
10	1			1

$$\text{Out} = \overline{B}\overline{C}\overline{D} + \overline{A}\overline{C}D + BCD + AC\overline{D}$$

$$\text{Out} = \overline{A}\overline{B}\overline{C} + \overline{A}BD + ABC + A\overline{B}\overline{D}$$

$$\text{Out} = \bar{C} + ABCD$$

		CD			
		00	01	11	10
A B	00	1	1		
	01	1	1		
	11	1	1	1	
	10	1	1		

$$\text{Out} = \bar{C} + ABD$$

Simplification by Boolean Algebra

$$\text{Out} = \bar{C} + ABCD$$

Applying rule $A + \bar{A}B = A + B$ to the $\bar{C} + ABCD$ term

$$\text{Out} = \bar{C} + ABD$$

$$\text{Out} = \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}C\overline{D} + \overline{A}B\overline{C}\overline{D} + \overline{A}BC\overline{D} + A\overline{B}\overline{C}\overline{D} \\ + A\overline{B}C\overline{D} + ABC\overline{D} + A\overline{B}C\overline{D} + A\overline{B}C\overline{D}$$

		CD			
		00	01	11	10
A/B	00	1		1	
	01	1		1	
	11	1	1	1	
	10	1		1	

$$\text{Out} = \overline{C}\overline{D} + CD + A\overline{B}\overline{C}$$

		CD			
		00	01	11	10
A/B	00	1		1	
	01	1		1	
	11	1	1	1	
	10	1		1	

$$\text{Out} = \overline{C}\overline{D} + CD + ABD$$

$$\text{Out} = \overline{A}\overline{B}C\overline{D} + \overline{A}B\overline{C}\overline{D} + A\overline{B}C\overline{D} + A\overline{B}C\overline{D} + A\overline{B}C\overline{D} + A\overline{B}C\overline{D} + A\overline{B}C\overline{D}$$

		CD			
		00	01	11	10
A/B	00			1	
	01			1	
	11	1	1	1	1
	10			1	

$$\text{Out} = AB + CD$$

CD \ AB	00	01	11	10
00			1	1
01	x	1	x	
11		1	x	
10	1	1		x

$$BD + AC'D' + A'CD'$$

K-map Simplification of POS Expressions

- Maxterms are grouped to find minimal PoS expression

		yz			
		00	01	11	10
x	0	$x + y + z$	$x + y + z'$	$x + y' + z'$	$x + y' + z$
	1	$x' + y + z$	$x' + y + z'$	$x' + y' + z'$	$x' + y' + z$

K-map Simplification of POS Expressions

- $F(W,X,Y,Z) = \prod M(0,1,2,4,5)$

		00	01	yz	11	10
x	0	$x + y + z$	$x + y + z'$	$x + y' + z'$	$x + y' + z$	
	1	$x' + y + z$	$x' + y + z'$	$x' + y' + z'$	$x' + y' + z$	

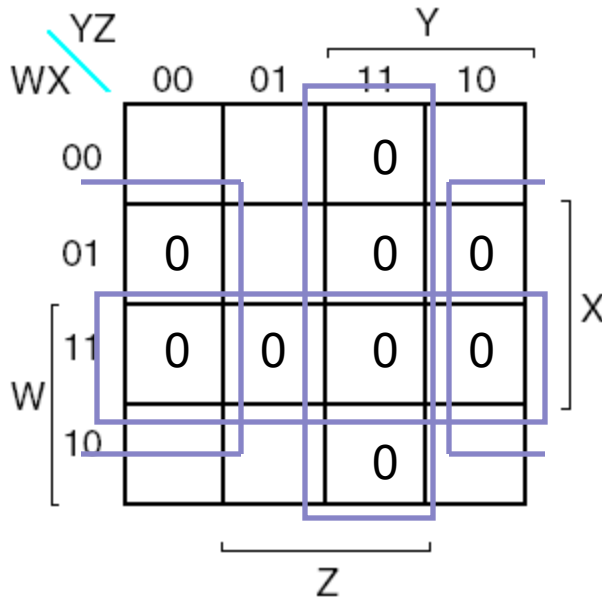
$$F(W,X,Y,Z) = Y \cdot (X + Z)$$

		00	01	yz	11	10
0		0	0		1	0
1		0	0		1	1

PoS Optimization from SoP

$$F(W,X,Y,Z) = \sum m(0,1,2,5,8,9,10)$$

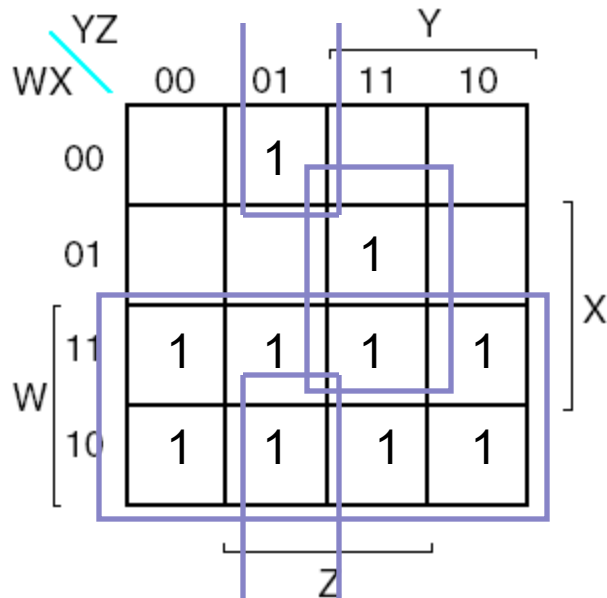
$$= \prod M(3,4,6,7,11,12,13,14,15)$$



$$F(W,X,Y,Z) = (W' + X')(Y' + Z')(X' + Z)$$

SoP Optimization from PoS

$$F(W,X,Y,Z) = \prod M(0,2,3,4,5,6) \\ = \sum m(1,7,8,9,10,11,12,13,14,15)$$



$$F(W,X,Y,Z) = W + XYZ + X'Y'Z$$

K-map Simplification of POS Expressions

AB \ CD	A+B	A+B'	A'+B'	A'+B
C+D	0	1	1	0
C+D'	0	1	0	0
C'+D'	0	1	1	1
C'+D	0	1	1	0

Diagram illustrating the K-map simplification of a POS expression. The K-map is a 5x5 grid with variables A, B, C, and D. The top row labels are AB, A+B, A+B', A'+B', and A'+B. The left column labels are CD, C+D, C+D', C'+D', and C'+D. The grid contains 0s and 1s. Three groups are identified: 1 (a vertical group of 0s in the first column), 2 (a horizontal group of 0s in the first row), and 3 (a horizontal group of 0s in the third row). Solid curved lines group the 0s in the first column and the 0s in the first row. A dashed curved line groups the 0s in the third row.

$$P = (A+B)(B+D)(A'+C+D')$$

$$\begin{aligned}
 \text{Out} = & \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}C\bar{D} \\
 & + \bar{A}B\bar{C}\bar{D} + \bar{A}B\bar{C}D + \bar{A}BCD \\
 & + AB\bar{C}\bar{D} + AB\bar{C}D + ABCD
 \end{aligned}$$

A \ B		CD			
		00	01	11	10
00	00	1	1	1	
01	00	1	1	1	
11	00	1	1	1	
10	00				

A \ B		CD			
		00	01	11	10
00	00	1	1	1	0
01	00	1	1	1	0
11	00	1	1	1	0
10	00	0	0	0	0

A \ B		CD			
		00	01	11	10
00	00	1	1	1	0
01	00	1	1	1	0
11	00	1	1	1	0
10	00	0	0	0	0

$$\text{Out} = \bar{A}\bar{C} + \bar{A}D + B\bar{C} + BD$$

$$\text{Out} = (\bar{A} + B)(\bar{C} + D)$$

Code converter

BCD to Excess-3 Code Converter

- BCD or Binary-Coded Decimal is a special kind of representation of a decimal number, where each individual digit of a number is converted into its equivalent 4 digit binary number and combining them all to generate a BCD code.
- For example, $(10)_{10} = (0001\ 0000)_{\text{BCD}}$
- BCD is not a binary representation of a decimal number.
- $(10)_{10} = (0001\ 0000)_{\text{BCD}} = (1010)_2$
- Invalid BCD:
 - In the BCD, with four bits we can represent sixteen numbers (0000 to 1111). But in BCD code only first ten of these are used (0000 to 1001).
 - The remaining six code combinations i.e.(1010 to 1111) are invalid in BCD.

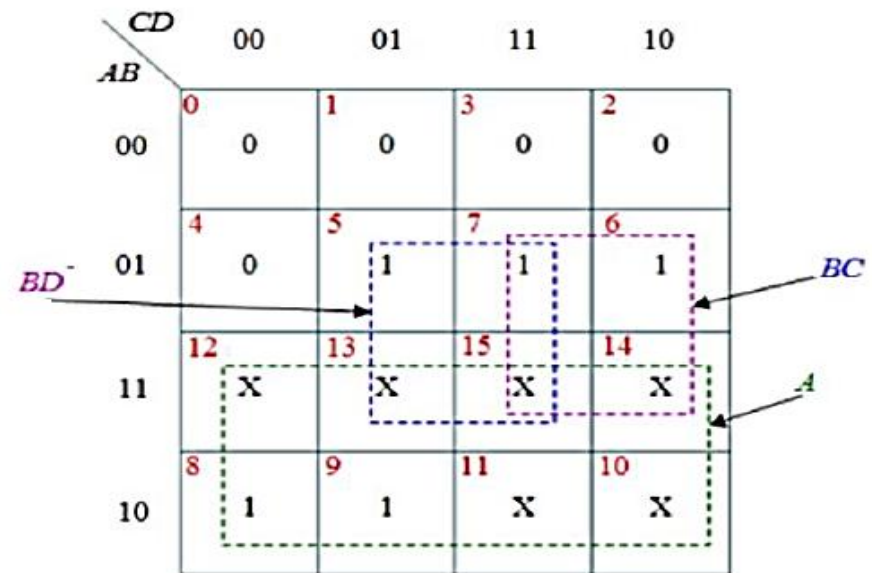
BCD to Excess-3 Code Converter

- A BCD digit can be converted to its corresponding Excess-3 code by simply adding 3 to it.
- Since we have only 10 digits(0 to 9) in decimal, we don't care about the rest and marked them with a cross(X).

BCD to Excess-3 Code Converter

BCD(8421)				Excess-3			
A	B	C	D	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	X	X	X	X
1	0	1	1	X	X	X	X
1	1	0	0	X	X	X	X
1	1	0	1	X	X	X	X
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X

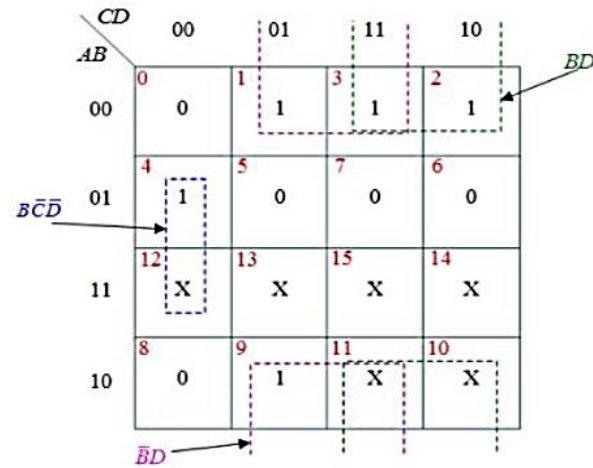
k-map for W



$$W = A + BC + BD$$

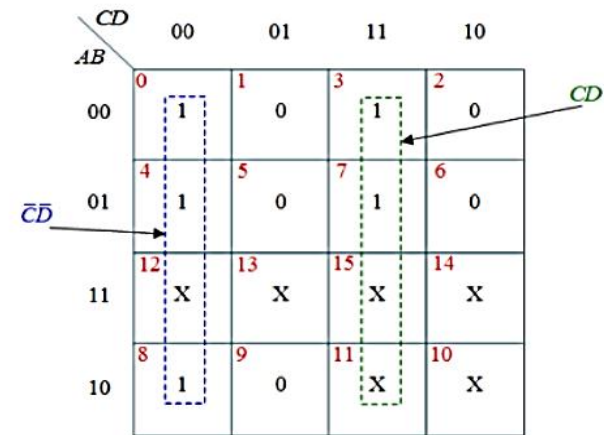
BCD(8421)				Excess-3			
A	B	C	D	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	X	X	X	X
1	0	1	1	X	X	X	X
1	1	0	0	X	X	X	X
1	1	0	1	X	X	X	X
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X

k-map for X



$$X = \bar{B}C + \bar{B}D + B\bar{C}\bar{D}$$

k-map for Y



$$Y = CD + \bar{C}\bar{D}$$

BCD(8421)				Excess-3			
A	B	C	D	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	X	X	X	X
1	0	1	1	X	X	X	X
1	1	0	0	X	X	X	X
1	1	0	1	X	X	X	X
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X

k-map for Z

		CD			
		00	01	11	10
AB	00	0 1	1 0	3 0	2 1
	01	4 1	5 0	7 0	6 1
	11	12 X	13 X	15 X	14 X
	10	8 1	9 0	11 X	10 X

$\bar{D}$ (indicated by a green arrow pointing to the 10 column)

$$Z = \bar{D}$$

BCD to Excess-3 Code Converter

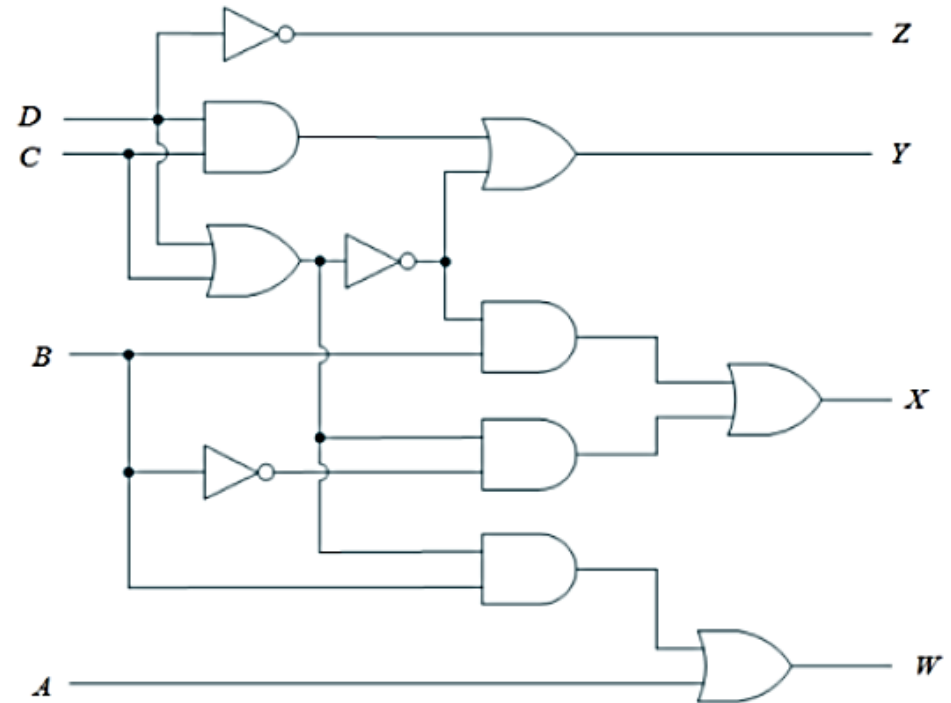
$$W = A + BC + BD$$

$$X = \bar{B}C + \bar{B}D + B\bar{C}\bar{D}$$

$$Y = CD + \bar{C}\bar{D}$$

$$Z = \bar{D}$$

Logic Circuit Diagram



Excess-3 to BCD Code Converter

Excess-3				BCD			
w	x	y	z	A	B	C	D
0	0	0	0	X	X	X	X
0	0	0	1	X	X	X	X
0	0	1	0	X	X	X	X
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	1
0	1	0	1	0	0	1	0
0	1	1	0	0	0	1	1
0	1	1	1	0	1	0	0
1	0	0	0	0	1	0	1
1	0	0	1	0	1	1	0
1	0	1	0	0	1	1	1
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	1
1	1	0	1	X	X	X	X
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X

K-Map for A

		yz			
		00	01	11	10
wx	00	X	X	0	X
	01	0	0	0	0
	11	1	X	X	X
	10	0	0	1	0

$$A = wx + wyz$$

K-Map for B

wx \ yz	00	01	11	10
00	X	X	0	X
01	0	0	1	0
11	0	X	X	X
10	1	1	0	1

K-Map for C

wx \ yz	00	01	11	10
00	X	X	0	X
01	0	1	0	1
11	0	X	X	X
10	0	1	0	1

K-Map for D

wx \ yz	00	01	11	10
00	X	X	0	X
01	1	0	0	1
11	1	X	X	X
10	1	0	0	1

$$B = x'y' + x'z' + xyz$$

$$C = y'z + yz'$$

$$D = z'$$

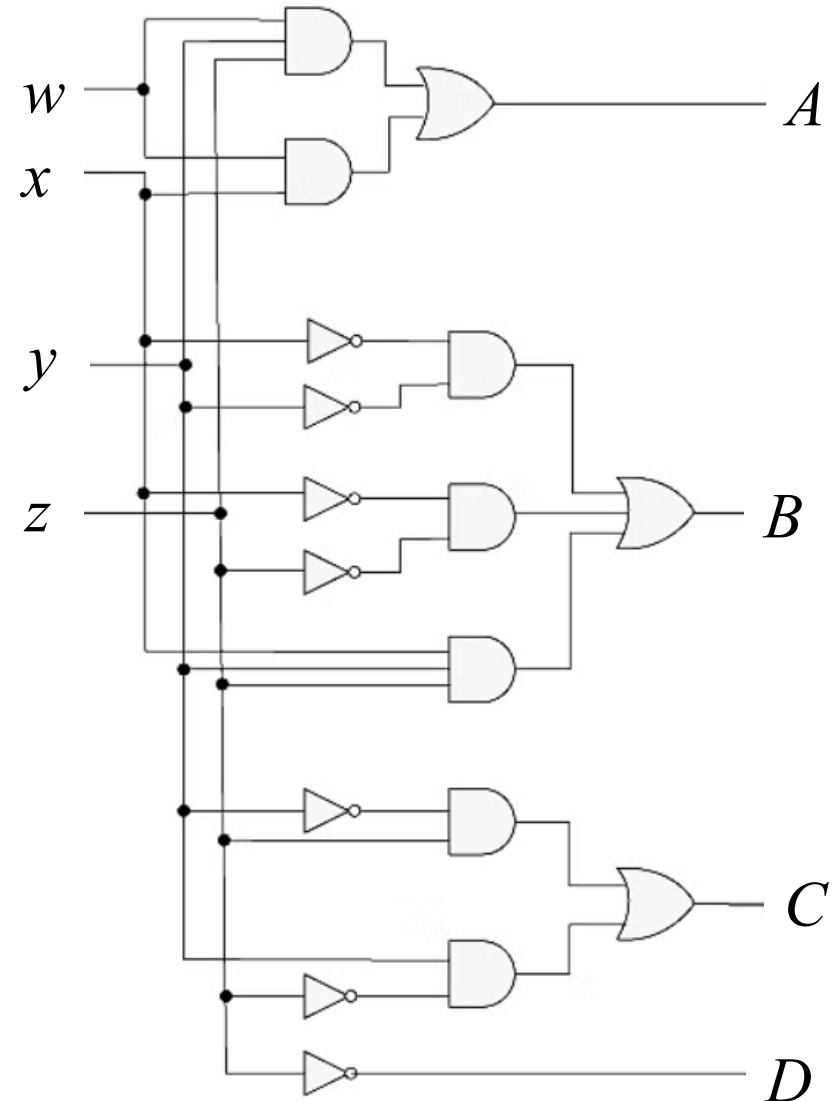
Excess-3 to BCD Code Converter

$$A = wx + wyz$$

$$B = x'y' + x'z' + xyz$$

$$C = y'z + yz'$$

$$D = z'$$



Binary to Gray Code Converter

- The Gray code is unweighted and it exhibits only a single bit change from one code word to the next in sequence.
- **Binary to Gray code conversion:**
- 1. The MSB in the Gray code is the same as the corresponding MSB in the binary number.
- 2. Going from left to right, add each adjacent pair of binary code bits to get the next Gray code bit. Discard carries.
- Example: Convert binary number 10110 to Gray code.

1	+	0	+	1	+	1	+	0	Binary
↓		↓		↓		↓		↓	
1		1		1		0		1	Gray

- The Gray code is 11101.

Binary to Gray code converter

Truth Table:

Binary				Gray Code			
b ₃	b ₂	b ₁	b ₀	g ₃	g ₂	g ₁	g ₀
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

K map for g₀ :

		b ₁ ,b ₀			
		00	01	11	10
b ₃ ,b ₂	00	0	1	0	1
	01	0	1	0	1
	11	0	1	0	1
	10	0	1	0	1

$$g_0 = b_0b'_1 + b_1b'_0 = b_0 \oplus b_1$$

K map for g_1 :

b3,b2 \ b1,b0					
		00	01	11	10
00	0	0	1	1	
01	1	1	0	0	
11	1	1	0	0	
10	0	0	1	1	

$$g_1 = b_2 b'_1 + b_1 b'_2 = b_1 \oplus b_2$$

K map for g_2 :

b3,b2 \ b1,b0					
		00	01	11	10
00	0	0	0	0	
01	1	1	1	1	
11	0	0	0	0	
10	1	1	1	1	

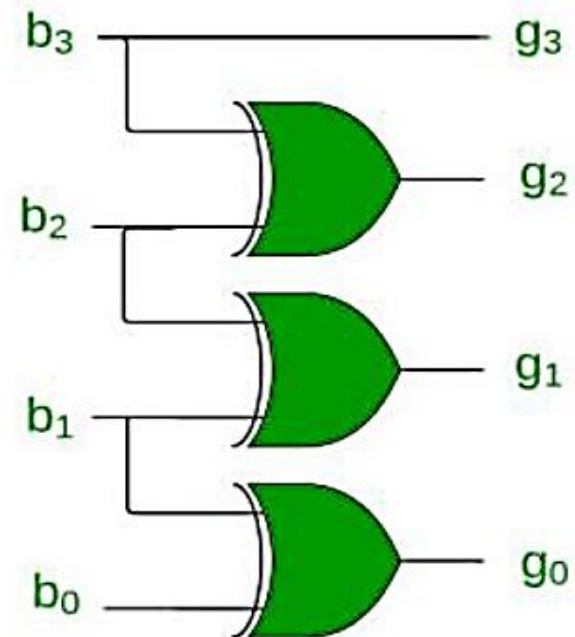
$$g_2 = b_2 b'_3 + b_3 b'_2 = b_2 \oplus b_3$$

K map for g_3 :

		b1,b0			
		00	01	11	10
b3,b2	00	0	0	0	0
	01	0	0	0	0
	11	1	1	1	1
	10	1	1	1	1

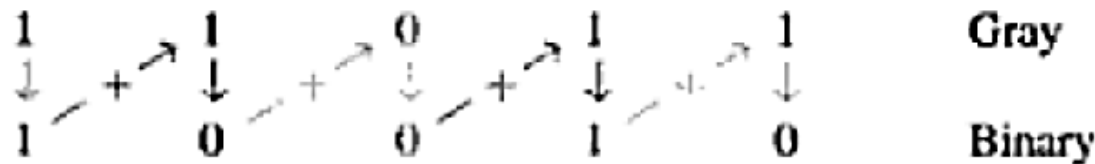
$$g_3 = b_3$$

Logic Diagram:



Gray to Binary Code Converter

- 1. The MSB in the binary code is the same as the corresponding bit in the Gray code.
- 2. Add each binary code bit generated to the Gray code bit in the next adjacent position. Discard the carries.
- Example: Convert the Gray code word 11011 to binary.



- The binary number is 10010.

Gray to Binary code converter

Truth Table:

Gray Code				Binary			
g_3	g_2	g_1	g_0	b_3	b_2	b_1	b_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	1
1	0	0	0	1	1	1	1
1	0	0	1	1	1	1	0
1	0	1	0	1	1	0	0
1	0	1	1	1	1	0	1
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	1
1	1	1	0	1	0	1	1
1	1	1	1	1	0	1	0

K map for b_3 :

		g_1, g_0			
		00	01	11	10
g_3, g_2	00	0	0	0	0
	01	0	0	0	0
	11	1	1	1	1
	10	1	1	1	1

$$b_3 = g_3$$

K map for b_2 :

		$g1, g0$			
		00	01	11	10
$g3, g2$	00	0	0	0	0
	01	1	1	1	1
	11	0	0	0	0
	10	1	1	1	1

$$\begin{aligned}
 b_2 &= g'_3 g_2 + g_3 g'_2 \\
 &= g_3 \oplus g_2
 \end{aligned}$$

K map for b_1 :

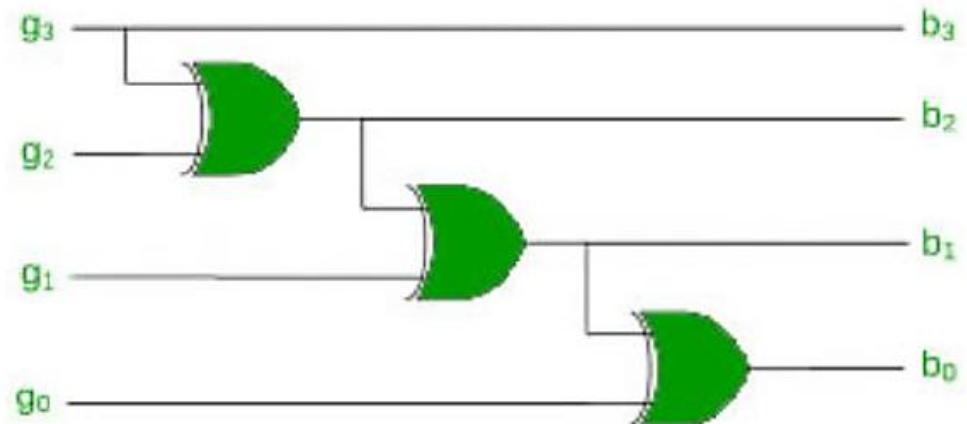
		$g1, g0$			
		00	01	11	10
$g3, g2$	00	0	0	1	1
	01	1	1	0	0
	11	0	0	1	1
	10	1	1	0	0

$$\begin{aligned}
 b_1 &= g'_3 g'_2 g_1 + g'_3 g_2 g'_1 + g_3 g_2 g_1 + g_3 g'_2 g'_1 \\
 &= g'_3 (g'_2 g_1 + g_2 g'_1) + g_3 (g_2 g_1 + g'_2 g'_1) \\
 &= g'_3 (g_2 \oplus g_1) + g_3 (g_2 \odot g_1) \\
 &= g_3 \oplus g_2 \oplus g_1
 \end{aligned}$$

K map for b_0 :

		g^1, g^0			
		00	01	11	10
g^3, g^2	00	0	1	0	1
	01	1	0	1	0
	11	0	1	0	1
	10	1	0	1	0

Logic Diagram:

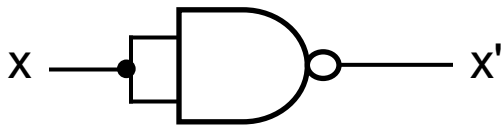


$$\begin{aligned}
 b_0 &= g'_3 g'_2 g'_1 g_0 + g'_3 g'_2 g_1 g'_0 + g'_3 g_2 g'_1 g'_0 + g'_3 g_2 g_1 g_0 + g_3 g'_2 g'_1 g'_0 + g_3 g'_2 g_1 g_0 \\
 &\quad + g_3 g_2 g'_1 g_0 + g_3 g_2 g_1 g'_0 \\
 &= g'_3 g'_2 (g'_1 g_0 + g_1 g'_0) + g'_3 g_2 (g'_1 g'_0 + g_1 g_0) + g_3 g'_2 (g'_1 g'_0 + g_1 g_0) \\
 &\quad + g_3 g_2 (g'_1 g_0 + g_1 g'_0) \\
 &= g'_3 g'_2 (g_0 \oplus g_1) + g'_3 g_2 (g_0 \odot g_1) + g_3 g'_2 (g_0 \odot g_1) + g_3 g_2 (g_0 \oplus g_1) \\
 &= (g_0 \oplus g_1)(g_2 \odot g_3) + (g_0 \odot g_1)(g_2 \oplus g_3) \\
 &= g_3 \oplus g_2 \oplus g_1 \oplus g_0
 \end{aligned}$$

Universal Gates: NAND and NOR

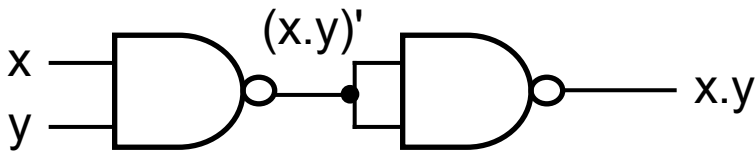
NAND Gate

- NAND gate is **self-sufficient** (can build any logic circuit with it).
- Can be used to implement AND/OR/NOT.
- Implementing an inverter using NAND gate:



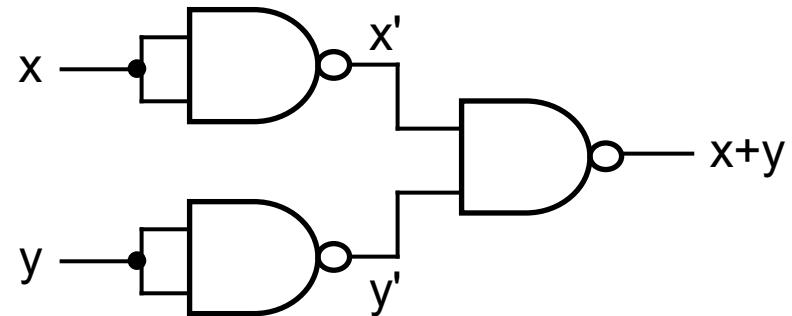
$$(x.x)' = x' \quad (\text{T1: idempotency})$$

- Implementing AND using NAND gates:



$$\begin{aligned} ((xy)'(xy)')' &= ((xy)')' && \text{idempotency} \\ &= (xy) && \text{involution} \end{aligned}$$

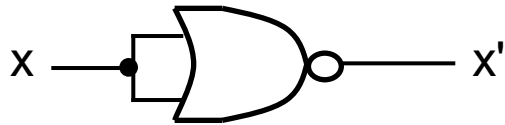
- Implementing OR using NAND gates:



$$\begin{aligned} ((xx)'(yy)')' &= (x'y')' && \text{idempotency} \\ &= x''+y'' && \text{DeMorgan} \\ &= x+y && \text{involution} \end{aligned}$$

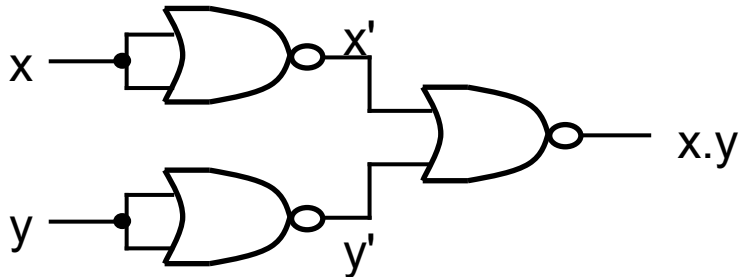
NOR Gate

- NOR gate is also self-sufficient.
- Can be used to implement AND/OR/NOT.
- Implementing an inverter using NOR gate:



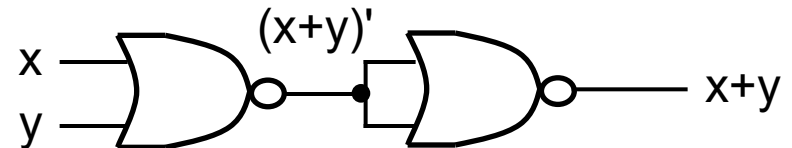
$$(x+x)' = x' \quad (\text{idempotency})$$

- Implementing AND using NOR gates:



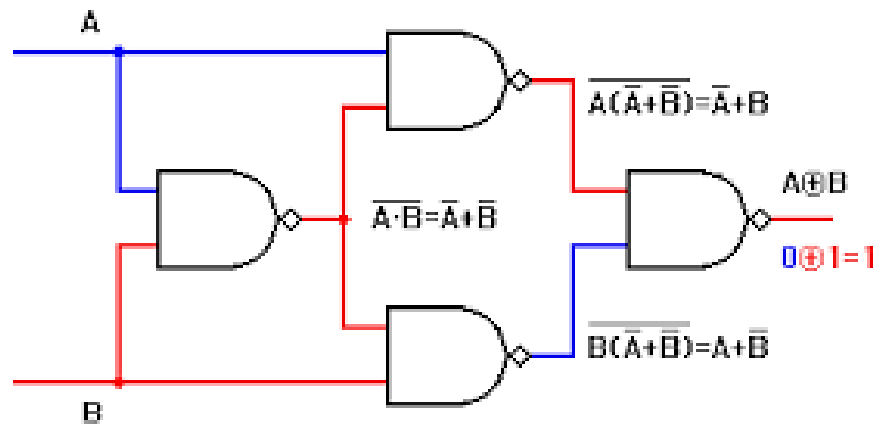
$$\begin{aligned} ((x+x)' + (y+y'))' &= (x' + y')' && \text{idempotency} \\ &= x'' \cdot y'' && \text{DeMorgan} \\ &= x \cdot y && \text{involution} \end{aligned}$$

- Implementing OR using NOR gates:

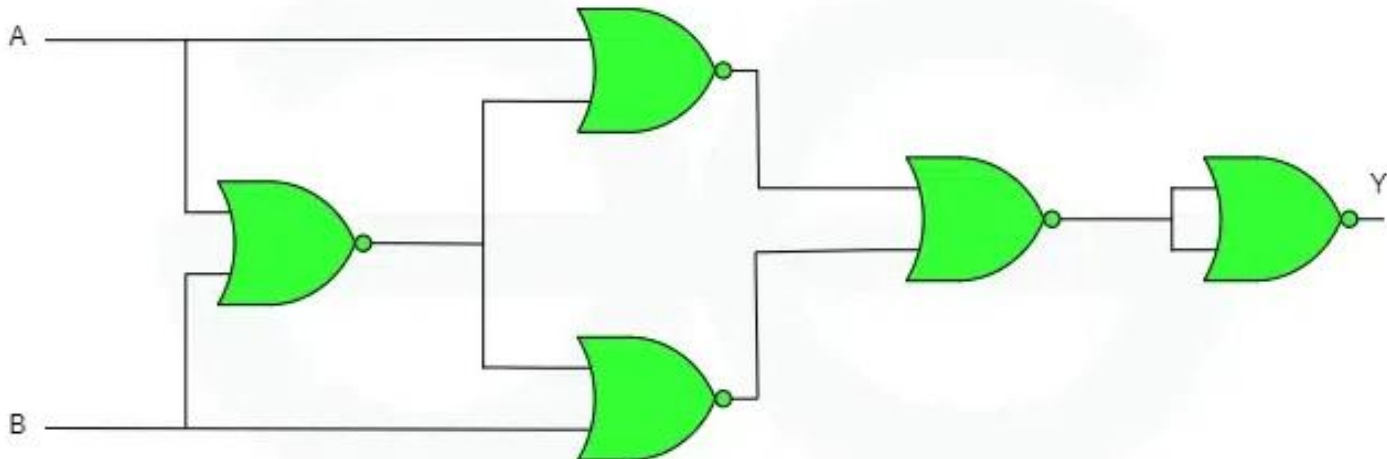


$$\begin{aligned} ((x+y)' + (x+y)')' &= ((x+y)')' && \text{idempotency} \\ &= (x+y) && \text{involution} \end{aligned}$$

Implement XOR gate with NAND gate



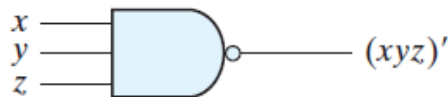
Implement XOR gate with NOR gate



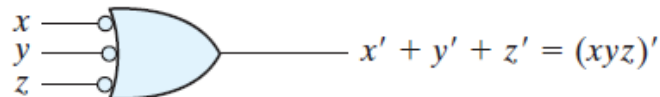
NAND AND NOR IMPLEMENTATION

NAND Circuits

- A convenient way to implement a Boolean function with NAND gates is to obtain the simplified Boolean function in terms of Boolean operators and then convert the function to NAND logic.



(a) AND-invert



(b) Invert-OR

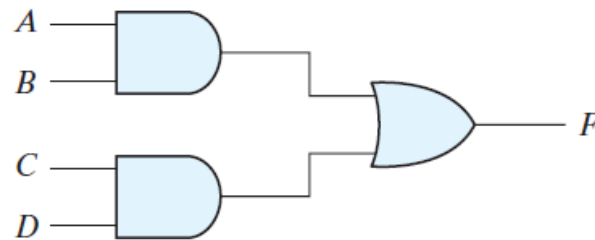
STEPS TO GET NAND - NAND LOGIC

- STEP 1: Draw the **logic circuit using AND, OR and NOT GATES**
- STEP 2: Put Bubbles at **output of AND and input of OR**.
- Step 3: Insert **Inverter (NOT GATE) for each bubbles**.
- Step 4: Remove **double NOT gates** connected in series (adjacent), if any
- Step 5: Replace **bubbled OR and Inverter** by NAND.

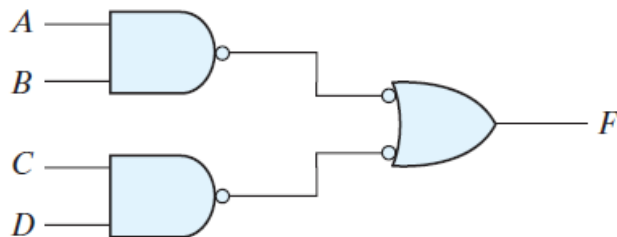
Two-Level Implementation

The implementation of Boolean functions with NAND gates requires that the functions be in sum-of-products form.

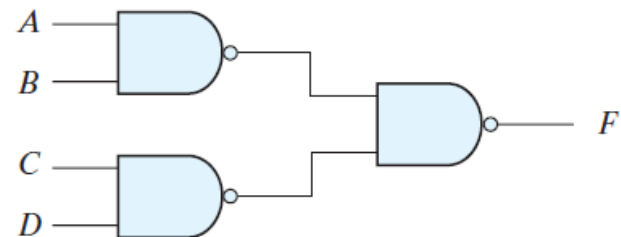
$$F = AB + CD$$



(a)



(b)

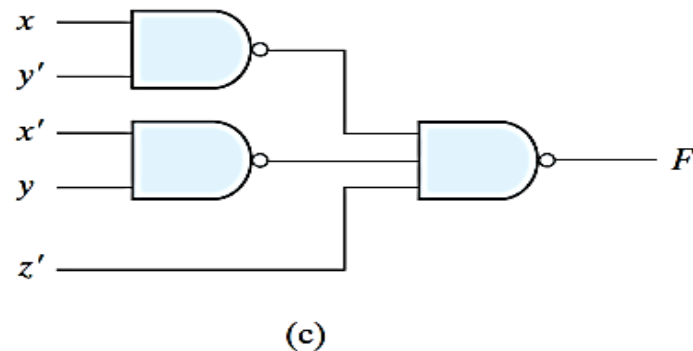
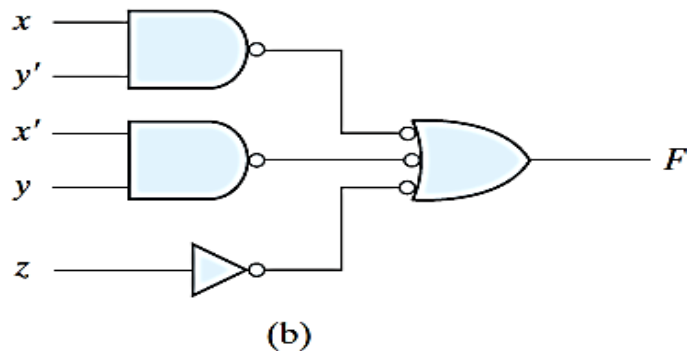
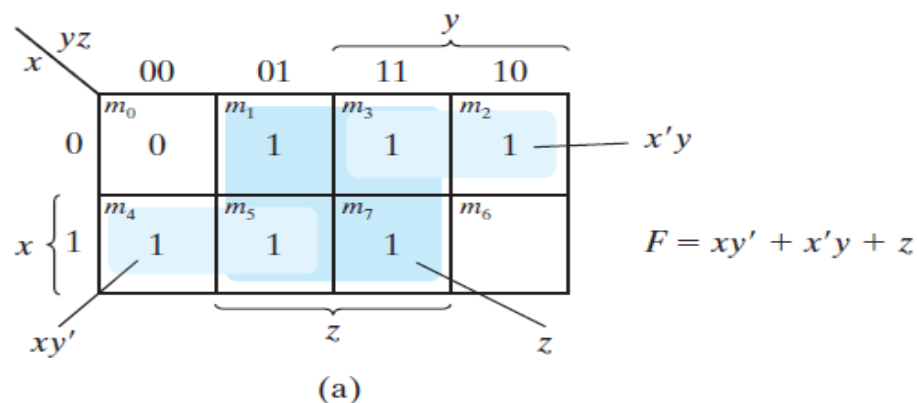


(c)

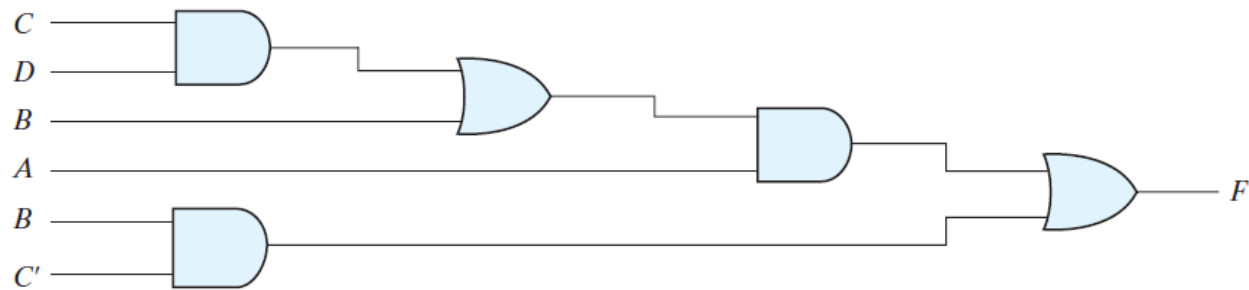
Implement the following Boolean function with NAND gates:

$$F(x, y, z) = (1, 2, 3, 4, 5, 7)$$

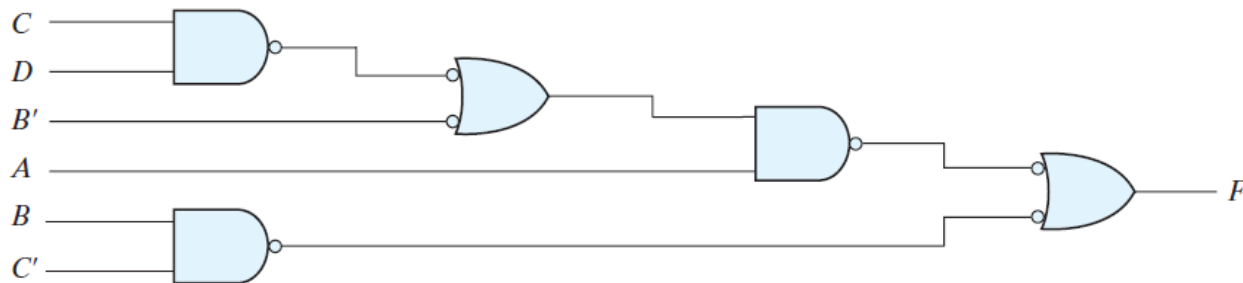
The first step is to simplify the function into sum-of-products form.



Implementing $F = A(CD + B) + BC'$

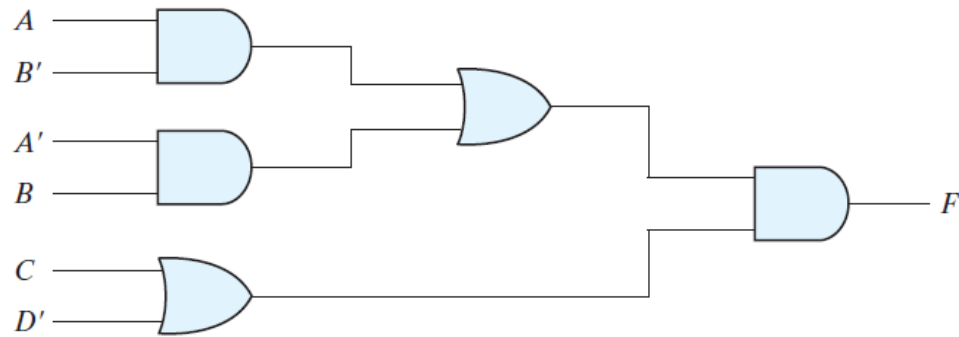


(a) AND-OR gates

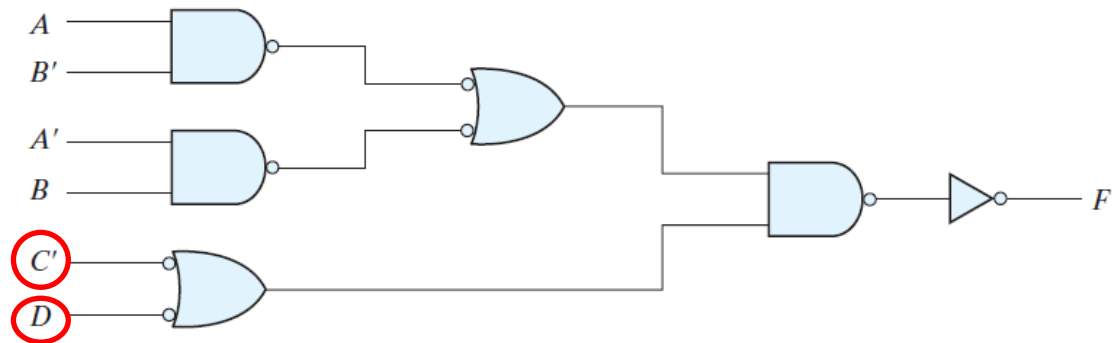


(b) NAND gates

$$F = (AB' + A'B)(C + D')$$

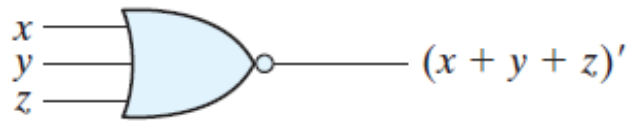


(a) AND-OR gates

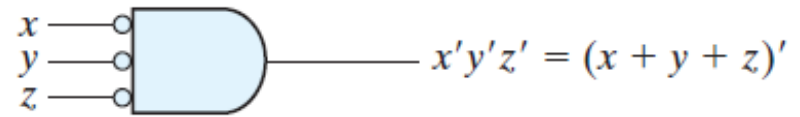


(b) NAND gates

NOR Implementation



(a) OR-invert



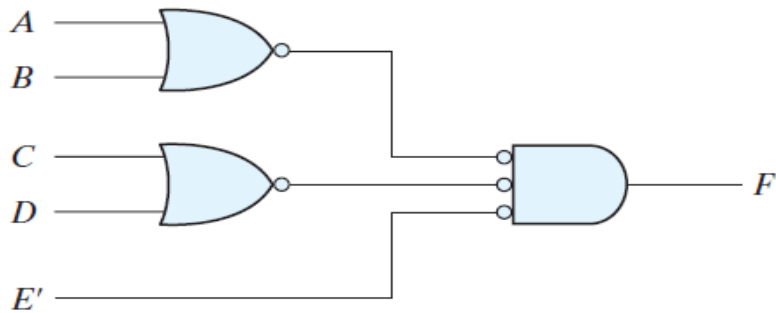
(b) Invert-AND

STEPS TO GET NOR- NOR LOGIC

- STEP 1: Draw the **logic circuit using AND, OR and NOT GATES**
- STEP 2: Put Bubbles at **output of OR** and **input of AND**
- Step 3: Insert **Inverter (NOT GATE) for each bubbles**.
- Step 4: Remove **double NOT gates** connected in series (adjacent), if any
- Step 5: Replace **bubbled AND and Inverter** by **NOR**.

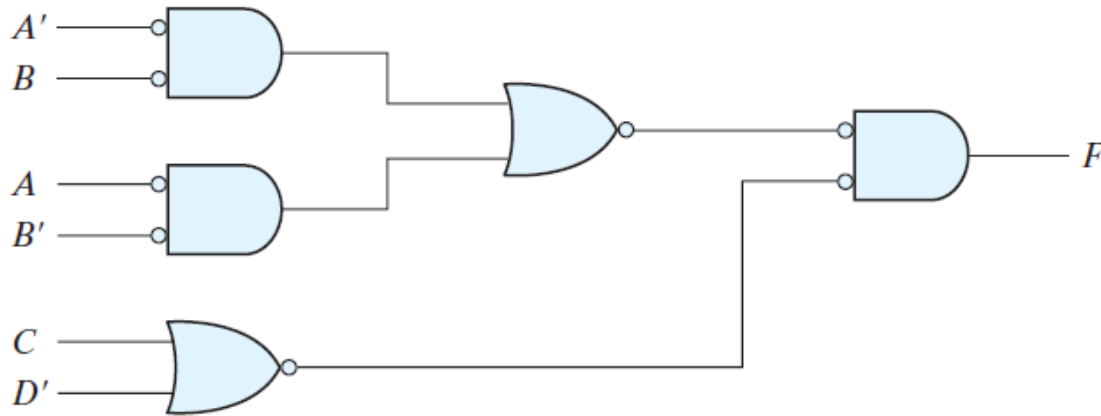
NOR Implementation

Implementing $F = (A + B)(C + D)E$



NOR Implementation

Implementing $F = (AB' + A'B)(C + D')$ with NOR gates



Practice Questions

2.2 Simplify the following Boolean expressions to a minimum number of literals:

(a)* $xy + xy'$

(b)* $(x + y)(x + y')$

(c)* $xyz + x'y + xyz'$

(d)* $(A + B)'(A' + B')'$

2.3 Simplify the following Boolean expressions to a minimum number of literals:

(a)* $ABC + A'B + ABC'$

(b)* $x'yz + xz$

(c)* $(x + y)'(x' + y')$

(d)* $xv + x(wz + wz')$

(e)* $(BC' + A'D)(AB' + CD')$

2.4 Reduce the following Boolean expressions to the indicated number of literals:

(a)* $A'C' + ABC + AC'$

to three literals

(b)* $(x'y' + z)' + z + xy + wz$

to three literals

(c)* $A'B(D' + C'D) + B(A + A'CD)$

to one literal

(d)* $(A' + C)(A' + C')(A + B + C'D)$

to four literals

2.4 (c)

$A'B(D'+C'D)+B(A+A'CD)$ TO ONE LITERAL

$$= B[A'(D'+C'D)+A+A'CD]$$

$$= B[A'(D'+C'D)+(A+A').(A+CD)]$$

$$= B[A'(D'+C'D)+A+CD]$$

$$= B[(A'+A)(D'+C'D+A)+CD]$$

$$= B[D'+C'D+A+CD]$$

$$= B[D'+C'D+A+CD]$$

$$= B[D'+D+A]$$

$$= B$$

2.4 (d)

$$(A'+C)(A'+C')(A+B+C'D)$$

$$(A'+C)(A'+C')=A'+CC'=A'+0=A'$$

$$A' \cdot (A+B+C'D)$$

$$= A'B+A'C'D$$

$$= A'(B+C'D) \quad A'(B+C'D)$$

Answers

2.2 (a) x (b) x (c) y (d) 0

2.3 (a) B (b) $z(x + y)$ (c) $x'y'$ (d) $x(w + y)$ (e) 0

2.4 (a) $AB + C'$ (b) $x + y + z$ (c) B (d) $A'(B + C'D)$

Simplification of Boolean Functions

Simplify the Boolean function $F = ((XY' + XYZ)' + X(Y + XY'))'$

$$\begin{aligned} F &= ((XY' + XYZ)' + X(Y + XY'))' \\ &= ((X(Y' + YZ))' + XY + XY')' \\ &= ((X(Y'Z + Y' + YZ))' + X(Y + Y'))' && (Y' = Y'(Z + 1) = Y'Z + Y') \\ &= (X(Y' + Z))' + X' \\ &= (X' + (Y' + Z)' + X)' \\ &= (1 + YZ)' \\ &= 1' \\ &= 0 \end{aligned}$$

Simplification of Boolean Functions

- Simplify the Boolean function $F = XYZ + XY'Z + XYZ'$.

$$F = XYZ + XY'Z + XYZ'$$

$$= XZ(Y + Y') + XYZ'$$

$$= XZ + XYZ'$$

$$= X(Z + YZ')$$

$$= X(Z + Y)$$

Simplify the Boolean function $F = A'B'C + A'BC + AB'$.

$$\begin{aligned} F &= A'B'C + A'BC + AB' \\ &= A'C (B' + B) + AB' \\ &= A'C + AB' \end{aligned}$$

Simplify the Boolean function $F = AB + (AC)' + AB'C(AB + C)$.

$$\begin{aligned} F &= AB + (AC)' + AB'C(AB + C) \\ &= AB + A' + C' + AB'C.AB + AB'C.C \\ &= AB + A' + C' + 0 + AB'C && (B.B' = 0 \text{ and } C.C = C) \\ &= ABC + ABC' + A' + C' + AB'C && (AB = AB(C + C') = ABC + ABC') \\ &= AC(B + B') + C'(AB + 1) + A' \\ &= AC + C' + A' && (B + B' = 1 \text{ and } AB + 1 = 1) \\ &= AC + (AC)' \\ &= 1 \end{aligned}$$

Practice questions

- Find the complement of $F = wx + yz$; then show that $FF' = 0$ and $F + F' = 1$
- Find the complement of the following expressions:

(a) $xy' + x'y$

(b) $(AB' + C)D' + E$

(c) $(x + y' + z)(x' + z')(x + y)$

- List the truth table of the function:

$$F = xy + xy' + y'z$$

- Logical operations can be performed on strings of bits by considering each pair of corresponding bits separately (this is called bitwise operation). Given two 8-bit strings $A = 10101101$ and $B = 10001110$, evaluate the 8-bit result after the following logical operations: (a) AND, (b) OR, (c) XOR, (d) NOT A , (e) NOT B .

Practice questions

- Draw the logic diagrams for the following Boolean expressions:

(a) $Y = A'B' + B(A + C)$ (b) $Y = BC + AC'$

(c) $Y = A + CD$ (d) $Y = (A + B)(C' + D)$

- Simplify the Boolean function T_1 and T_2 to a minimum number of literals.

A	B	C	T_1	T_2
0	0	0	1	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	0	1
1	0	1	0	1
1	1	0	0	1
1	1	1	0	1

Practice questions

- Given the Boolean function

$$F = xy'z + x'y'z + w'xy + wx'y + wxy$$

- (a) Obtain the truth table of the function.
- (b) Draw the logic diagram using the original Boolean expression.
- (c) Simplify the function to a minimum number of literals using Boolean algebra. $F = y'z + y(w + x)$
- (d) Obtain the truth table of the function from the simplified expression and show that it is the same as the one in part (a)
- (e) Draw the logic diagram from the simplified expression and compare the total number of gates with the diagram of part (b).

Practice questions

- Convert the following to the other canonical form:

(a) $F(x, y, z) = \sum (1, 3, 7)$

(b) $F(A, B, C, D) = \prod (0, 1, 2, 3, 4, 6, 12)$

- Convert the following expressions into sum of products and product of sums:

(a) $(AB + C)(B + C'D)$

(b) $x' + x(x + y')(y + z')$

Practice questions

Simplify the following Boolean functions, using three-variable maps:

(a) $F(x, y, z) = \Sigma(0, 2, 4, 5)$

(b) $F(x, y, z) = \Sigma(0, 2, 4, 5, 6)$

(c) $F(x, y, z) = \Sigma(0, 1, 2, 3, 5)$

(d) $F(x, y, z) = \Sigma(1, 2, 3, 7)$

Simplify the following Boolean functions, using three-variable maps:

(a)* $F(x, y, z) = \Sigma(0, 1, 5, 7)$

(b)* $F(x, y, z) = \Sigma(1, 2, 3, 6, 7)$

(c) $F(x, y, z) = \Sigma(2, 3, 4, 5)$

(d) $F(x, y, z) = \Sigma(1, 2, 3, 5, 6, 7)$

(e) $F(x, y, z) = \Sigma(0, 2, 4, 6)$

(f) $F(x, y, z) = \Sigma(3, 4, 5, 6, 7)$

Simplify the following Boolean expressions, using three-variable maps:

(a)* $xy + x'y'z' + x'yz'$

(b)* $x'y' + yz + x'yz'$

(c)* $F(x, y, z) = x'y + yz' + y'z'$

(d) $F(x, y, z) = x'yz + xy'z' + xy'z$

Simplify the following Boolean functions, using *Karnaugh* maps:

$$(a)^* F(x, y, z) = \Sigma(2, 3, 6, 7)$$

$$(b)^* F(A, B, C, D) = \Sigma(4, 6, 7, 15)$$

$$(c)^* F(A, B, C, D) = \Sigma(3, 7, 11, 13, 14, 15)$$

$$(d)^* F(w, x, y, z) = \Sigma(2, 3, 12, 13, 14, 15)$$

$$(e) F(w, x, y, z) = \Sigma(11, 12, 13, 14, 15)$$

$$(f) F(w, x, y, z) = \Sigma(8, 10, 12, 13, 14)$$

Simplify the following Boolean functions, using four-variable maps:

$$(a)^* F(w, x, y, z) = \Sigma(1, 4, 5, 6, 12, 14, 15)$$

$$(b) F(A, B, C, D) = \Sigma(2, 3, 6, 7, 12, 13, 14)$$

$$(c) F(w, x, y, z) = \Sigma(1, 3, 4, 5, 6, 7, 9, 11, 13, 15)$$

$$(d)^* F(A, B, C, D) = \Sigma(0, 2, 4, 5, 6, 7, 8, 10, 13, 15)$$

Simplify the following Boolean expressions, using four-variable maps:

$$(a)^* A'B'C'D' + AC'D' + B'CD' + A'BCD + BC'D$$

$$(b)^* x'z + w'xy' + w(x'y + xy')$$

$$(c) A'B'C'D + AB'D + A'BC' + ABCD + AB'C$$

$$(d) A'B'C'D' + BC'D + A'C'D + A'BCD + ACD'$$

Simplify the following Boolean expressions, using four-variable maps:

$$(a)^* w'z + xz + x'y + wx'z$$

$$(b) AD' + B'C'D + BCD' + BC'D$$

$$(c)^* AB'C + B'C'D' + BCD + ACD' + A'B'C + A'BC'D$$

$$(d) wxy + xz + wx'z + w'x$$

Find the minterms of the following Boolean expressions by first plotting each function in a map:

(a)* $xy + yz + xy'z$

(b)* $C'D + ABC' + ABD' + A'B'D$

(c) $wyz + w'x' + wxz'$

(d) $A'B + A'CD + B'CD + BC'D'$

Find all the prime implicants for the following Boolean functions, and determine which are essential:

(a)* $F(w, x, y, z) = \Sigma(0, 2, 4, 5, 6, 7, 8, 10, 13, 15)$

(b)* $F(A, B, C, D) = \Sigma(0, 2, 3, 5, 7, 8, 10, 11, 14, 15)$

(c) $F(A, B, C, D) = \Sigma(2, 3, 4, 5, 6, 7, 9, 11, 12, 13)$

(d) $F(w, x, y, z) = \Sigma(1, 3, 6, 7, 8, 9, 12, 13, 14, 15)$

(e) $F(A, B, C, D) = \Sigma(0, 1, 2, 5, 7, 8, 9, 10, 13, 15)$

(f) $F(w, x, y, z) = \Sigma(0, 1, 2, 5, 7, 8, 10, 15)$

Simplify the following Boolean functions by first finding the essential prime implicants:

(a) $F(w, x, y, z) = \Sigma(0, 2, 5, 7, 8, 10, 12, 13, 14, 15)$

(b) $F(A, B, C, D) = \Sigma(0, 2, 3, 5, 7, 8, 10, 11, 14, 15)$

(c)* $F(A, B, C, D) = \Sigma(1, 3, 4, 5, 10, 11, 12, 13, 14, 15)$

(d) $F(w, x, y, z) = \Sigma(0, 1, 4, 5, 6, 7, 9, 11, 14, 15)$

(e) $F(A, B, C, D) = \Sigma(0, 1, 3, 7, 8, 9, 10, 13, 15)$

(f) $F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 7, 10, 15)$

Convert the following Boolean function from a sum-of-products form to a simplified product-of-sums form.

$$F(x, y, z) = \Sigma(0, 1, 2, 5, 8, 10, 13)$$

Simplify the following Boolean functions:

(a)* $F(A, B, C, D) = \Pi(1, 3, 5, 7, 13, 15)$

(b) $F(A, B, C, D) = \Pi(1, 3, 6, 9, 11, 12, 14)$

Simplify the following expressions to (1) sum-of-products and (2) products-of-sums:

(a)* $x'z' + y'z' + yz' + xy$

(b) $ACD' + C'D + AB' + ABCD$

(c) $(A' + B + D')(A' + B' + C')(A' + B' + C)(B' + C + D')$

(d) $BCD' + ABC' + ACD$

Give three possible ways to express the following Boolean function with eight or fewer literals:

$$F = A'BC'D + AB'CD + A'B'C' + ACD'$$

Simplify the following Boolean function F , together with the don't-care conditions d , and then express the simplified function in sum-of-minterms form:

(a) $F(x, y, z) = \Sigma(0, 1, 4, 5, 6)$

$d(x, y, z) = \Sigma(2, 3, 7)$

(b)* $F(A, B, C, D) = \Sigma(0, 6, 8, 13, 14)$

$d(A, B, C, D) = \Sigma(2, 4, 10)$

(c) $F(A, B, C, D) = \Sigma(5, 6, 7, 12, 14, 15,)$

$d(A, B, C, D) = \Sigma(3, 9, 11, 15)$

(d) $F(A, B, C, D) = \Sigma(4, 12, 7, 2, 10,)$

$d(A, B, C, D) = \Sigma(0, 6, 8)$

Draw a NAND logic diagram that implements the complement of the following function:

$$F(A, B, C, D) = \Sigma(0, 1, 2, 3, 6, 10, 11, 14)$$

Draw a logic diagram using only two-input NOR gates to implement the following function:

$$F(A, B, C, D) = (A \oplus B)'(C \oplus D)$$

Simplify the following functions, and implement them with two-level NOR gate circuits:

(a)* $F = wx' + y'z' + w'yz'$

(b) $F(w, x, y, z) = \Sigma(0, 3, 12, 15)$

(c) $F(x, y, z) = [(x + y)(x = z)]'$

Thank you